

课程设计报告

ROS2 小型四旋翼无人机仿真系统的设计与实现

Design and Implementation of a Lightweight Quadrotor Simulator Based on ROS2

姓名：_____ 学号：_____ 班级：_____

开发环境	Ubuntu 22.04、ROS2 Humble、C++17、Python 3
代码仓库	https://github.com/VMnicht/drone_sim
完成日期	2026 年 7 月

摘要

本项目完成了一套面向课程实验的小型四旋翼仿真系统。系统运行于 ROS2 Humble，核心动力学、控制器、电机分配器、轨迹、扰动和传感器数学模型集中在 `drone_core` 中，只依赖 Eigen 与 C++ 标准库。ROS2 节点负责参数装载、消息转换和可视化，地图、点云与规划模块则保留对 ROS2 的直接依赖。这样的边界使控制算法可以脱离通信框架单独构建和测试，也便于以后接入其他中间件或硬件在环环境。系统实现了六自由度刚体动力学、串级几何控制、三维体素 A*、局部点云、故障与风扰注入、多机仿真、RViz2 和 Web 地面站。11 个正式场景全部通过自动阈值验收。悬停稳态误差为 0.0203 m，三组避障实验的最小净空分别为 0.4196 m、0.4136 m 和 0.4889 m，均高于 0.30 m 的安全约束。实验表明，这套实现能够在较少外部依赖下形成从模型、控制到规划和评测的完整闭环，同时保留清晰的工程复现路径。

关键词：四旋翼；ROS2；六自由度动力学；几何控制；三维 A*；仿真评测

1 引言

四旋翼仿真器看起来只是把若干微分方程放进 ROS2 节点，真正实现时却很容易在接口和时间尺度上出问题。电机命令、刚体积分、控制器和可视化具有不同更新频率，任何一个环节的单位或时间戳处理不一致，都会表现为模型闪烁、姿态发散或轨迹无法收敛。因此，本项目没有直接包装现成仓库，而是从状态定义、力矩方向和消息数据链重新搭建仿真闭环，并用可重复的场景逐步验收。

课程要求以目标点为输入，以四路电机转速为控制输出，同时给出动力学、地图、避障、可视化和完整说明。项目在此基础上继续实现了风扰、传感器噪声、点云与体素感知、多机、解析轨迹、故障注入、地面站、参数扫描和自动评测。所有加分项都进入统一启动入口和正式实验，而不是作为互不相连的演示脚本存在。

设计中最重要取舍是让算法与 ROS2 解耦。drone_core 内保存动力学、模型控制、mixer、轨迹生成、扰动和噪声模型，只使用 Eigen 与 C++ STL。ROS2 适配层负责 topic、QoS、参数和坐标系。地图和规划需要直接消费 PointCloud2、Marker 与 Path，且这些模块的意义本来就与机器人中间件的数据组织紧密相关，因此允许它们保留 ROS2 依赖。这个边界既符合课程要求，也让核心算法可以用普通 CMake 和 CTest 独立验证。

2 系统总体设计

2.1 模块划分与数据流

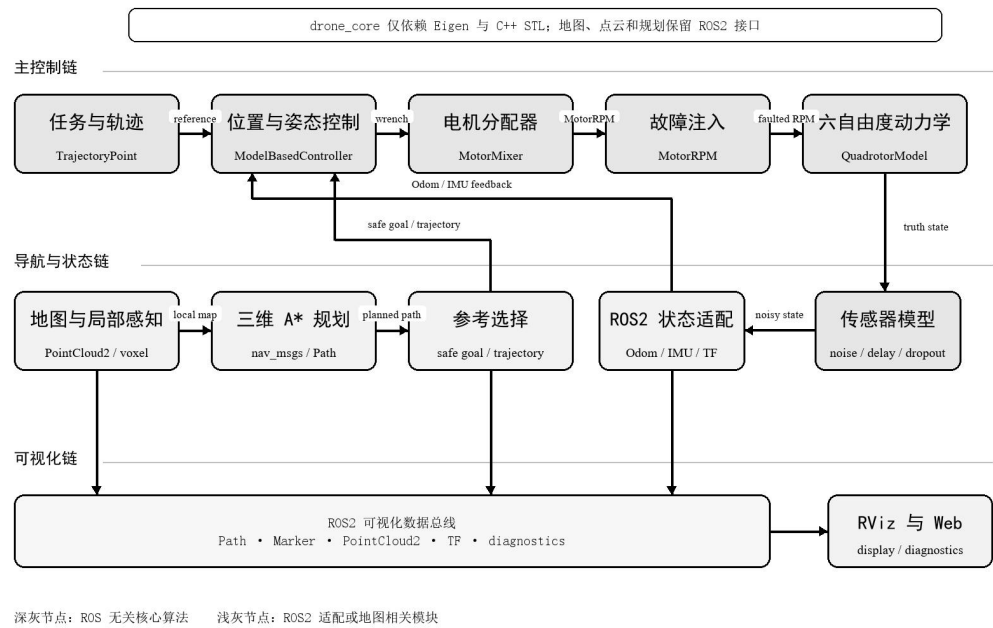


图 1 系统节点与主要数据流

系统的数据流可以概括为两个闭环。飞行闭环从任务或轨迹参考进入控制器，经 X 型电机分配和故障注入后驱动动力学，再由传感器模型把真值状态转换为控制器可用的 Odom 与 IMU。导航闭环从静态地图和局部点云出发，三维 A* 输出安全路径与局部目标，随后仍复用同一控制器。RViz2 与 Web 地面站只订阅状态和诊断信息，不参与动力学积分，因此界面退出不会改变实验结果。

环节	主要 ROS2 接口	消息类型	更新频率
动力学	/drone/motor_rpm_faulted → /drone/odom	MotorRPM / Odometry	约 100 Hz
控制	/drone/goal、/drone/odom → /drone/motor_rpm_cmd	PoseStamped / MotorRPM	约 100 Hz
导航	/map/obstacles、/drone/local_points → /drone/planned_path	MarkerArray / PointCloud2 / Path	事件触发与低频
显示	/tf、/drone/path、/drone/markers	TFMessage / Path / MarkerArray	1~5 Hz

表 1 主要节点接口与频率分工

2.2 工程组织与启动方式

工作区共有 15 个 ROS2 package。drone_msgs 定义电机转速、障碍物与轨迹点消息；drone_dynamics、drone_controller 和 drone_sensors 是核心算法的 ROS2 适配层；drone_map、drone_perception 与 drone_planner 负责环境和安全路径；其余 package 承担多机、故障、可视化、工具、地面站与 launch 编排。根目录的 start_sim.sh 会在脚本内部加载 ROS2 和工作区环境，用户不需要手动 source。若尚未构建，脚本先执行 colcon build，再按 hover、experiment、multi、ground-station 或 panel 模式启动。

3 四旋翼动力学模型

3.1 状态、输入与电机响应

动力学状态由世界系位置 $\mathbf{p}$ 、速度 $\mathbf{v}$ 、机体到世界的单位四元数 $\mathbf{q}$ 、机体系角速度 $\boldsymbol{\Omega}$ 以及四个电机实际角速度组成。控制消息以 RPM 表示，进入核心模型前换算为 rad/s。模型默认质量为 1.0 kg，X 型机臂长度为 0.17 m，惯量对角项为 0.02、0.02 和 0.04 kg·m²。电机没有被处理成理想瞬时执行器，而是使用时间常数 τ_m 的一阶响应。

$$\omega_{i,k+1} = \omega_{i,k} + [1 - \exp(-\Delta t / \tau_m)] (\omega_{i,\text{cmd}} - \omega_{i,k})$$

每次更新后都执行上下限裁剪。这样既能避免控制器给出的极端命令进入平方推力模型，也能重现起飞阶段转速建立的短暂过程。默认最大电机角速度为 2300 rad/s，对应约 21963 RPM。理论悬停角速度为 1133.15 rad/s，约 10821 RPM，与实验稳态曲线相符。

参数	符号	数值	说明
质量	m	1.0 kg	机体总质量
惯量	I	diag[0.02, 0.02, 0.04] kg·m ²	机体系对角惯量
机臂长度	l	0.17 m	质心到旋翼中心
推力系数	kF	1.91×10 ⁻⁶ N/(rad/s) ²	单旋翼平方推力
反扭矩系数	kM	2.60×10 ⁻⁷ N·m/(rad/s) ²	偏航反扭矩
重力加速度	g	9.81 m/s ²	世界系 z 轴向上

表 2 动力学默认参数

3.2 推力、力矩与刚体积分

第 i 个旋翼的推力为 $F_i = kF\omega_i^2$ ，反扭矩为 $M_i = s_i kM\omega_i^2$ 。s_i 由旋翼转向决定。四个旋翼按 X 型布局叠加为总推力 T 和三个轴向力矩 $\boldsymbol{\tau}$ 。平动方程在世界坐标系计算，转动方程在机体系计算，并加入线性阻力、角速度阻尼和可选外部扰动力。

$$\mathbf{m} \dot{\mathbf{v}} = \mathbf{R}(\mathbf{q}) [0, 0, T]^T - m \mathbf{g} \mathbf{e}_3 - \mathbf{C}\mathbf{v} + \mathbf{F}_{\text{ext}}$$

$$\mathbf{I} \dot{\boldsymbol{\Omega}} = \boldsymbol{\tau} - \boldsymbol{\Omega} \times \mathbf{I} \boldsymbol{\Omega} - \mathbf{C}\boldsymbol{\Omega} + \boldsymbol{\tau}_{\text{ext}}$$

$$\dot{\mathbf{q}} = 1/2 \cdot \mathbf{q} \otimes [0, \boldsymbol{\Omega}], \text{ 每步积分后执行 } \mathbf{q} \leftarrow \mathbf{q} / \|\mathbf{q}\|$$

数值积分使用固定步长的半隐式更新。角速度和线速度先由加速度推进，位置和四元数再使用新状态更新。地面附近加入非穿透约束，防止数值误差让机体落到 z 轴负半空间。仿真真值只由动力学节点产生一次 Odom 和 TF，传感器节点发布带噪副本，避免多个发布者导致 RViz 中同一模型交替跳变。

4 控制器设计

前馈项来自解析轨迹或规划器；反馈项来自带噪 Odom 和 IMU

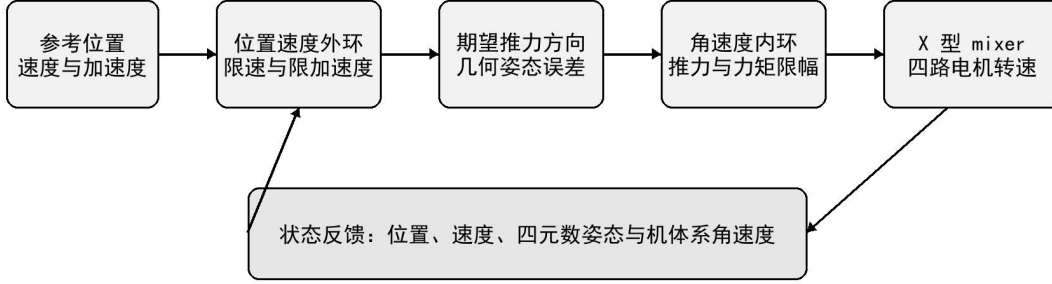


图 2 串级控制器与状态反馈关系

4.1 位置外环与期望姿态

控制器采用位置、速度外环和几何姿态内环。给定参考位置 p_d 、速度 v_d 、加速度 a_d 及期望偏航角，外环先计算位置误差 $e_p = p - p_d$ 与速度误差 $e_v = v - v_d$ ，再得到期望加速度。逐轴增益来自 `controller.yaml`，位置积分项仅在限定范围内累积，并设置反饱和回退。

$$a_c = a_d - K_p \odot e_p - K_v \odot e_v - K_i \odot \int e_p dt$$

期望合力 $F_d = m(a_c + g e_3)$ 。其方向确定期望机体 z 轴，再结合期望偏航构造完整旋转矩阵 R_d 。相较于直接把位置误差线性换算成 `roll` 和 `pitch`，这种构造在三维轨迹中没有欧拉角奇异点，且能清楚地限制合力和最大倾角。外环还限制目标跳变、水平速度、垂直速度和加速度，避免距离很远时一步给出不可实现的姿态。

4.2 几何姿态内环与 mixer

姿态误差从 $R^T R_d - R_d^T R$ 的反对称部分提取，角速度误差由当前角速度与期望角速度之差得到。控制力矩同时补偿刚体陀螺耦合项 $\Omega \times I \Omega$ 。总推力取期望合力在当前机体 z 轴上的投影，因此姿态尚未对准时不会把不正确的水平分量直接当作竖直推力。

$$\tau_c = -K_R \odot e_R - K_\Omega \odot e_\Omega + \Omega \times I \Omega$$

`mixer` 解算 T 、 τ_x 、 τ_y 、 τ_z 到四个 ω_i^2 。解算结果先保证非负，再经过推力、力矩、电机角速度和变化率的多级限幅。控制节点将 `rad/s` 转为消息中的 `RPM`，动力学适配层再还原为角速度。故障节点位于两者之间，可以按时间窗口降低单个电机效率、限制最大转速，或模拟命令丢包、延迟和冻结。这样的串接方式让正常控制器代码不需要知道故障是否存在。

5 地图、感知与避障

5.1 地图与局部点云

静态地图由 `YAML` 中的 `box` 和 `cylinder` 描述，也可以使用固定随机种子生成障碍物。地图节点发布可视 `Marker` 和障碍物几何信息。感知节点对几何表面采样，再根据无人机位置与朝向执行量程、视场、遮挡、噪声和丢点处理，输出局部 `PointCloud2`。点云进一步体素化，并使用短时持久化与过期衰减减少边界抖动。由于地图表示和 `PointCloud2` 接口紧密相关，这部分保留 `ROS2` 依赖，没有强行移入 `drone_core`。

安全距离采用障碍物几何边界到机体中心的净空定义。规划时将机体半径与 `0.30 m` 安全裕度共同膨胀到障碍物上，因此搜索节点可以按点机器人处理。实验记录器独立计算实际轨迹到原始障碍物表面的最小净空，用来验证规划假设是否在闭环飞行中成立。

5.2 三维体素 A* 与路径跟踪

规划器把起点、目标点和膨胀障碍物投影到三维体素网格，在 6、18 或 26 邻接中搜索。代价由移动距离和启发函数组成。若起点或终点落入占据体素，节点先在有限邻域内寻找最近自由体素；若开放集耗尽、搜索节点数超过上限或无法重建父节点链，则明确报告失败，不向控制器发布穿越障碍物的直线路径。

得到离散路径后，先做共线点合并和带碰撞检测的线段简化，再由前视距离选择局部安全目标。路径跟踪进度只允许沿路径前进，避免折返或自交路径中前视点误跳到尚未经过的后段。`perception_replan` 场景会在局部点云发生有效变化时触发重规划，同时设置最小重规划周期，防止感知噪声造成频繁振荡。

5.3 可视化与交互

RViz2 中同时显示无人机机体、四个旋翼、目标点、历史轨迹、障碍物、局部点云、体素和规划路径。无人机 Marker 使用固定命名空间与稳定 ID，时间戳设为零并启用 `frame_locked`；地图类低频信息使用 `transient-local QoS`。此前模型闪烁的根因是 Odom、TF 和 Marker 存在重复发布者，且显示对象生命周期过短。修正后每类状态只保留一个权威发布者，Marker 不再周期性失效。`start_sim.sh` 在 WSL 内部处理 DISPLAY 与环境加载，因此用户可以直接运行脚本而不必手动 source。

Web 地面站用于发布目标、查看位置和高度曲线、启停扰动与故障。任务 Panel 汇总 30 个任务入口、11 个正式场景、16 类指标和 7 类结果图，并允许在白名单范围内修改 29 份 YAML。界面只写配置副本且保留回滚，避免误操作破坏仓库基线。

6 实验设计与结果

6.1 实验方法

正式实验统一从 `start_sim.sh` 调度。每个场景使用固定参数与随机种子，记录 `telemetry.csv`、`reference_history.csv`、`summary.json`、`rosvbag` 和七类曲线。评价指标包含最终误差、稳态误差、RMS 误差、到达时间、最大姿态倾角、电机饱和比例、扰动恢复时间和最小障碍物净空。验收阈值存放在 `evaluation.yaml`，验证脚本只读取阈值与结果，不在代码中另设一套宽松标准。

表 3 的数值来自当前 `artifacts/experiments` 目录，而不是按设计参数推算。RMS 误差包含从地面起飞和目标切换阶段，因此在多航点、阵风和避障场景中会明显高于最终误差。它更能反映整个任务过程，而稳态误差用于判断最终悬停质量，两者不能互相替代。

场景	最终误差/m	稳态误差/m	RMS 误差/m	关键结果
悬停	0.0132	0.0203	0.3409	通过
单目标点	0.0203	0.0201	0.7122	通过
正方形航点	0.0168	0.0219	0.5038	通过
圆轨迹	0.0424	0.0458	0.3827	通过
八字轨迹	0.0972	0.0847	0.2813	通过
阵风扰动	0.0457	0.0723	1.1929	恢复 2.223 s
传感器噪声	0.0777	0.0604	0.3292	通过
电机故障	0.0179	0.0305	1.3669	最大倾角 19.11°
五障碍物	0.0247	0.0189	0.6955	净空 0.4196 m
狭窄通道	0.0134	0.0191	1.1548	净空 0.4136 m
感知重规划	0.0135	0.0190	0.6567	净空 0.4889 m

表 3 11 个正式场景的实测结果

6.2 基础飞行与轨迹跟踪

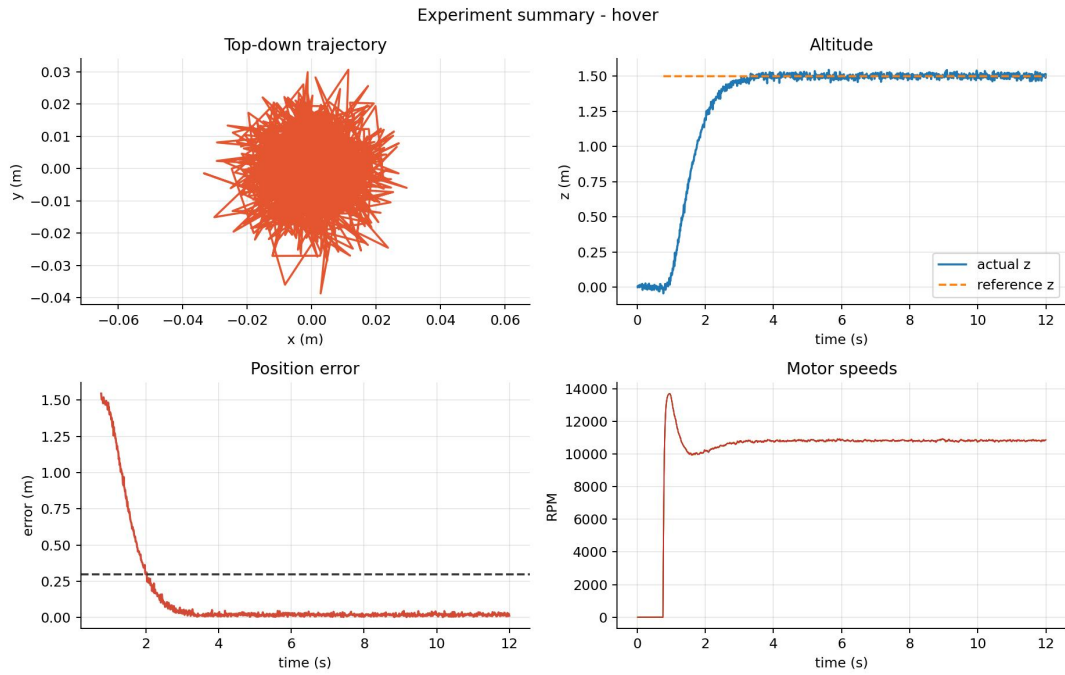


图3 悬停实验的轨迹、高度、位置误差和电机转速

悬停实验在约 1.99 s 首次进入 0.30 m 误差范围，随后高度稳定在 1.5 m 附近，稳态误差为 0.0203 m。四路电机在起飞瞬间共同升速，之后收敛到理论悬停转速附近；最大倾角仅 0.65° ，说明静态悬停没有依赖持续大姿态补偿。

正方形任务依次完成 5 个航点。每次参考点切换会让误差重新升高，但局部目标到达后均可继续收敛，最终误差为 0.0168 m。圆轨迹与 Gerono 八字轨迹分别得到 0.3827 m 和 0.2813 m 的全程 RMS 误差。解析轨迹提供速度、加速度和 yaw 前馈，因此连续轨迹没有被离散成一串急停目标。

6.3 扰动、噪声与故障

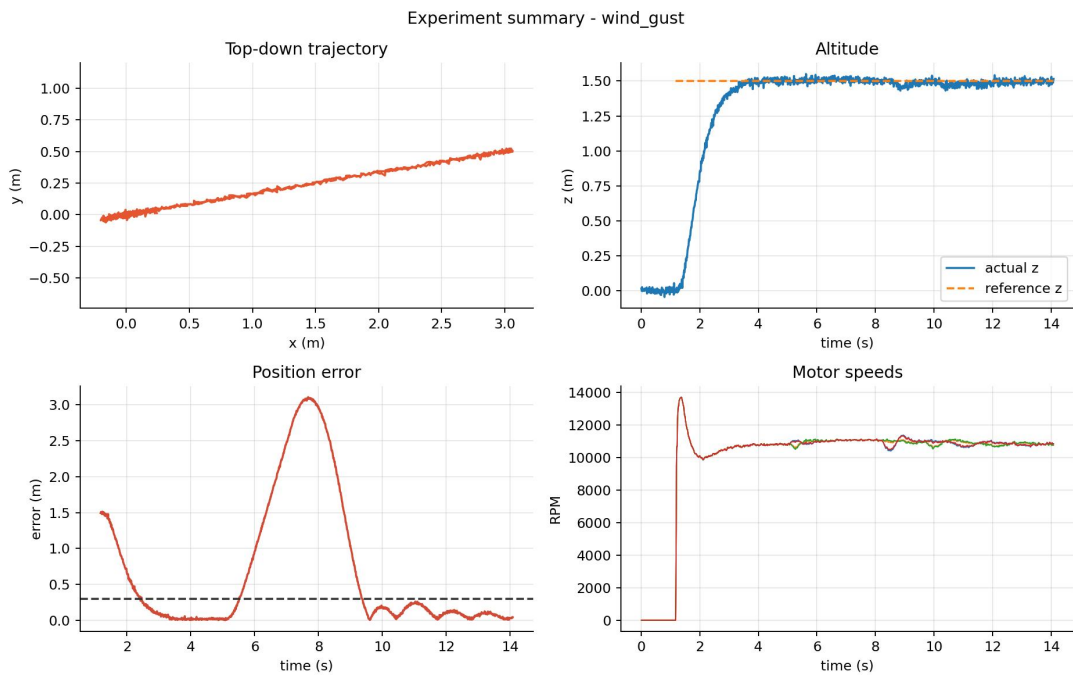


图4 阵风场景的偏差峰值与恢复过程

阵风场景在飞行中段施加峰值 3.041 N 的外力。位置误差先升至明显峰值，扰动结束后经过 2.223 s 回到验收区间，最终误差为 0.0457 m。传感器噪声场景把带噪 Odom 和 IMU 真正接入闭环，最终误差为 0.0777 m。电机故障场景在限定窗口降低 0 号电机效率，最大倾角增至 19.11°，故障解除后最终仍收敛至 0.0179 m。三类试验说明控制器具备恢复能力，但当前系统尚未包含故障辨识或自适应分配，因此不能把结果解释为对任意严重故障都鲁棒。

6.4 避障与局部感知

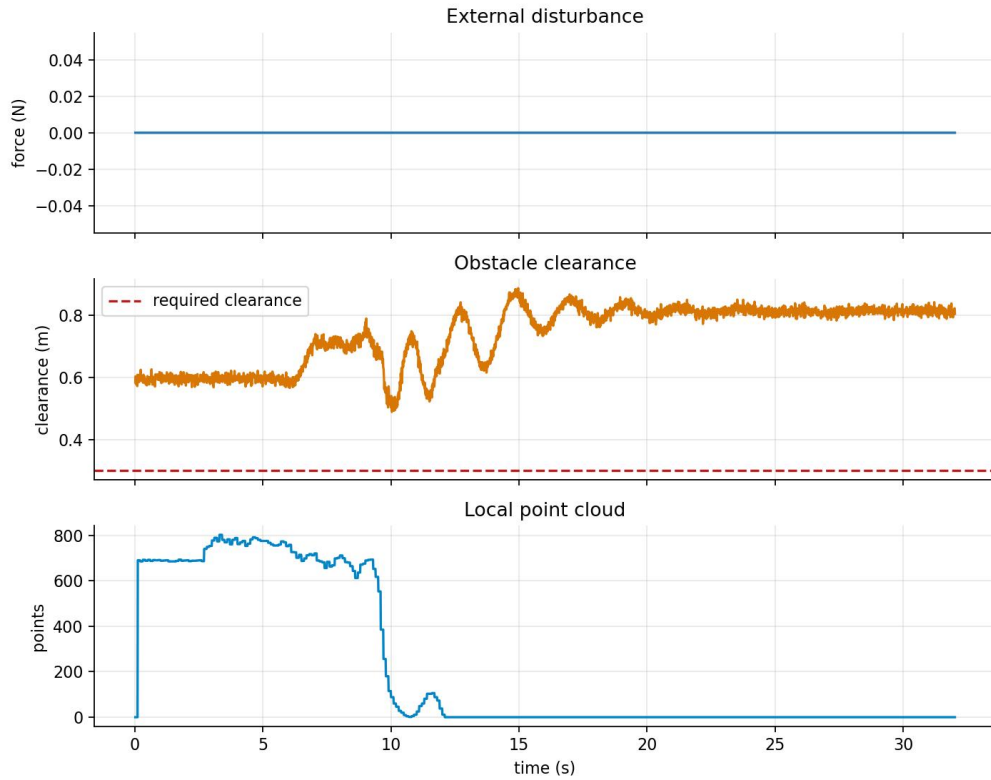


图 5 感知重规划场景的净空与局部点云规模

五障碍物、狭窄通道和感知重规划场景均到达目标。三者最小净空分别为 0.4196 m、0.4136 m 和 0.4889 m，高于 0.30 m 的安全合同。图 5 中局部点云随无人机接近障碍物上升到约 800 点，离开局部环境后逐渐归零；净空曲线在整个任务中没有越过红色阈值。规划路径是几何安全路径，实际轨迹还会受到控制超调和传感器噪声影响，因此使用真实轨迹净空作为最终判据。

7 加分功能与工程验证

课程列出的加分方向均已实现。参数方面，质量、惯量、电机系数、控制增益、传感器、地图、规划与场景均放入 YAML，并由脚本反向检查参数是否真正被节点声明和 launch 装载。扰动模块支持常值、正弦、阵风和固定种子随机力；传感器模块支持白噪声、偏置、随机游走、延迟与丢包；轨迹模块支持圆、八字和 waypoint list。

多机模式同时启动 3 套独立 namespace、TF、控制和传感器实例，fleet monitor 记录到的最小间距为 0.786 m，高于 0.75 m 要求，未发生违规。故障注入覆盖效率下降、转速上限、命令丢包、延迟和冻结。Web 地面站提供状态与控制入口，参数扫描对位置增益和风力尺度执行 3×3 实验，确定性回放则比较两次运行的轨迹和指标。

参数扫描显示，扰动力尺度是峰值误差的主要影响因素，位置比例增益在当前范围内只带来较小差异。这与单次手工调参相比更有解释力：它说明继续提高 K_p 并不能抵消外力增长，还可能增加姿态和电机余量压力。测试层面，工作区包含 gtest、launch test、Panel 进程测试、场景阈值检查、回放和多机专项验证。drone_core 还能使用 DRONE_CORE_STANDALONE=ON 脱离 ROS2 构建，直接运行 CTest。

8 与参考项目的关系

pengyu_sim 提供了教学型仿真工程的组织参考，尤其是将动力学、控制和可视化分成可独立理解的模块。MARSIM 更强调轻量级点云仿真，并把地图、局部感知和多机接口放到统一的 ROS 数据流中。本项目借鉴的是问题拆分方式和接口思想，没有复制其 package 或源码。当前工程以 ROS2 Humble 重新实现核心链路，并把可复现实验、自动验收和算法解耦放在更高优先级。

与 MARSIM 相比，本项目的环境真实感和激光雷达物理细节较弱，不包含 GPU 渲染或真实雷达扫描模式；优点是依赖轻、构建快，动力学和控制器可独立测试。与一般的完整 Gazebo 仿真相比，本项目也没有螺旋桨气动、机架柔性和碰撞接触模型。它更适合课程中验证控制、规划和 ROS2 工程边界，不适合作为高保真飞控认证工具。

9 失败案例与改进过程

开发中最难定位的问题不是公式本身，而是可视化和路径进度。早期 RViz 中 drone_model 会闪烁，Topic 状态在 OK 与 Error 之间跳变。检查发布者数量和 Marker 生命周期后发现，真值节点、传感器节点和可视化节点对相近状态重复发布，Marker 还使用了会过期的寿命。最终统一 Odom 与 TF 的权威来源，给 Marker 使用稳定 ID、零时间戳和 frame_locked，并把地图类信息改为 transient-local QoS。修正后 topic 频率稳定，模型不会因显示对象轮换而消失。

另一个问题出现在折返和自交路径。若只寻找离无人机最近的路径点，当前位置可能被投影到尚未走过的后半段，规划器就会误判进度并提前发布终点。现在的实现保存单调前进的路径索引，只在有限前向窗口内寻找最近点，再沿弧长选择前视目标。这个改动通过专门的回归测试覆盖，不依赖某一张地图恰好不触发问题。

这些失败也暴露出当前限制。固定步长模型无法描述高速飞行中的复杂气动，点云来自几何表面采样而非真实光束模型，A* 的计算量会随体素分辨率快速增加。故障恢复主要依靠原控制器余量，严重单电机失效仍可能超出可控范围。报告中的成功结果只对应已记录参数和场景，不应外推到未验证的边界。

10 AI 辅助过程与验证责任

本项目使用 AI 帮助梳理任务清单、生成部分代码骨架、补充测试思路和定位 ROS2 配置问题。关键公式、坐标系、旋翼方向、消息单位和安全阈值均由人工结合代码与曲线复核。ai_usage.md 保存了超过 8 条关键交互摘要，也记录了 AI 曾给出的错误建议及修正过程。

验证时没有把“能够编译”当作正确。动力学用悬停转速、自由响应和四元数归一化测试；控制器检查静态平衡、力矩符号、限幅与 mixer 输出；ROS2 层检查 topic 类型、发布者数量、频率、TF 连续性和 QoS；系统层则以 11 个场景的 CSV、JSON、PNG 与 rosbag 作为证据。AI 负责提高迭代速度，但判断模型是否可信、实验是否满足阈值以及报告是否如实描述，仍由项目实现者承担。

11 总结与后续工作

本项目完成了从四路电机输入、六自由度刚体动力学、串级控制到地图感知、三维规划、RViz2 和自动评测的闭环。核心算法与 ROS2 的边界清楚，地图相关模块保留中间件依赖；一键脚本解决了 WSL 下反复 source 和显示环境配置的问题。11 个正式场景全部通过，悬停、连续轨迹、风扰、噪声、故障和避障都有可复核的原始数据。

如果继续投入两周，第一优先级是给动力学加入更可靠的螺旋桨气动、地效和接触模型，并用真实飞控日志标定电机时间常数和阻力。第二步是把局部感知替换为可配置的光束扫描模型，引入动态障碍物和增量式规划。第三步是建立持续集成中的无头场景回归，固定发布频率、CPU 占用和误差基线。完成这些工作后，这套课程仿真器可以进一步用于控制器参数研究、规划算法对比和轻量级硬件在环验证。

参考文献

- [1] potato77. pengyu_sim: UAV simulation reference project[EB/OL]. https://gitee.com/potato77/pengyu_sim.
- [2] Zhang J, Xu W, Zhu Y, et al. MARSIM: A Light-weight Point-realistic Simulator for LiDAR-based UAVs[EB/OL]. arXiv:2211.10716, 2022.
- [3] Lee T, Leok M, McClamroch N H. Geometric Tracking Control of a Quadrotor UAV on SE(3)[C]//49th IEEE Conference on Decision and Control. 2010: 5420-5425.
- [4] Quigley M, Conley K, Gerkey B, et al. ROS: An Open-source Robot Operating System[C]//ICRA Workshop on Open Source Software. 2009.
- [5] Open Robotics. ROS 2 Humble Documentation[EB/OL]. <https://docs.ros.org/en/humble/>.
- [6] VMnicht. drone_sim: ROS2 小型四旋翼仿真器[EB/OL]. https://github.com/VMnicht/drone_sim, 2026.

附录 A 复现实验与材料位置

本报告中的数值均可从仓库现有材料反向核对。复核时先在工作区根目录执行 `./start_sim.sh batch` 重新生成场景，再运行 `python3 scripts/verify_experiments.py` 检查阈值。若只审阅已有结果，可直接查看下表所列文件，不需要启动 RViz2。

材料	仓库位置	用途
正式实验	<code>artifacts/experiments/<scenario>/</code>	CSV、JSON、PNG 与 rosbag
模型参数	<code>src/drone_bringup/config/</code>	动力学、控制、地图与验收阈值
算法说明	<code>docs/dynamics.md</code> 、 <code>docs/controller.md</code>	公式、单位与接口约定
AI 使用记录	<code>ai_usage.md</code>	关键交互、错误与人工复核
演示视频	<code>output/video/drone_demo.mp4</code>	1~3 分钟功能演示

表 4 报告结论的可复核材料

仓库公开地址为 https://github.com/VMnicht/drone_sim。启动脚本会自行加载 ROS2 和工作区环境，WSL 用户无需在终端中重复 `source`。若图形界面不可用，所有场景仍可用无头模式运行并生成同样的统计文件。